ResearchX

Intelligent Research Discovery & Project Similarity Platform

1. Problem & Solution

Problem: Students and researchers often struggle to discover relevant research papers, identify projects that are too similar to an existing idea, organize paper–author relationships, and locate potential research gaps. Keyword-only search can miss papers whose meaning is related but whose wording is different.

Solution: ResearchX combines a relational database with vector similarity search. Structured information about users, projects, domains, papers and authors is stored in MySQL, while semantic embeddings of project descriptions and research text are referenced in a vector database such as Qdrant. Users can search papers, analyze project similarity, receive related research results and explore structured research relationships.

2. System Workflow

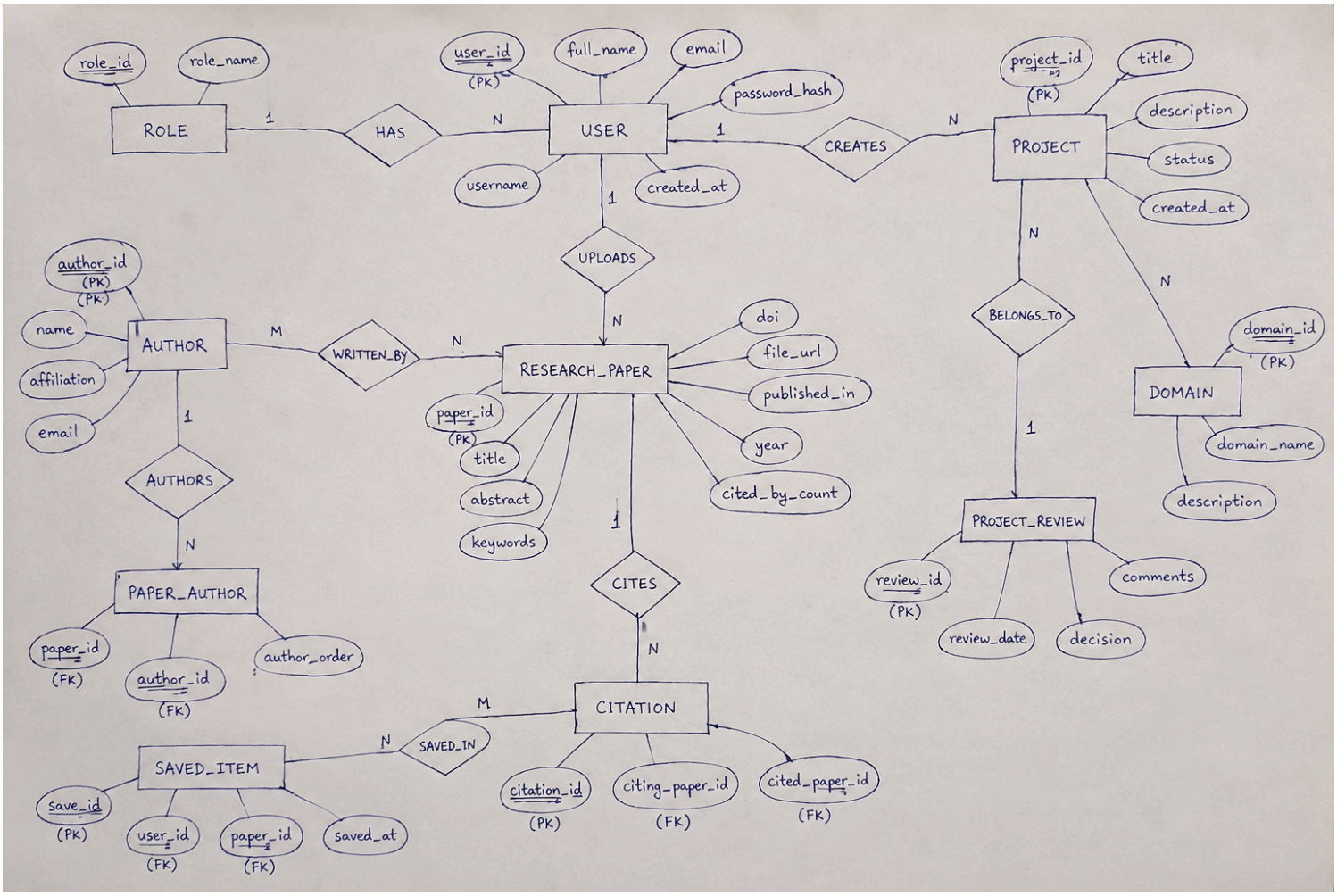
1	Input	Student enters a project idea, problem statement or research topic.
2	Process	The application prepares the text and generates an embedding for semantic comparison.
3	Store	Structured records go to MySQL; vector references and embeddings go to Qdrant.
4	Search	Researchers use full-text search or semantic similarity to retrieve relevant papers and similar projects.
5	Explore	Next.js displays matches, similarity scores, research relationships and analytics.

3. Technology Stack & Architecture

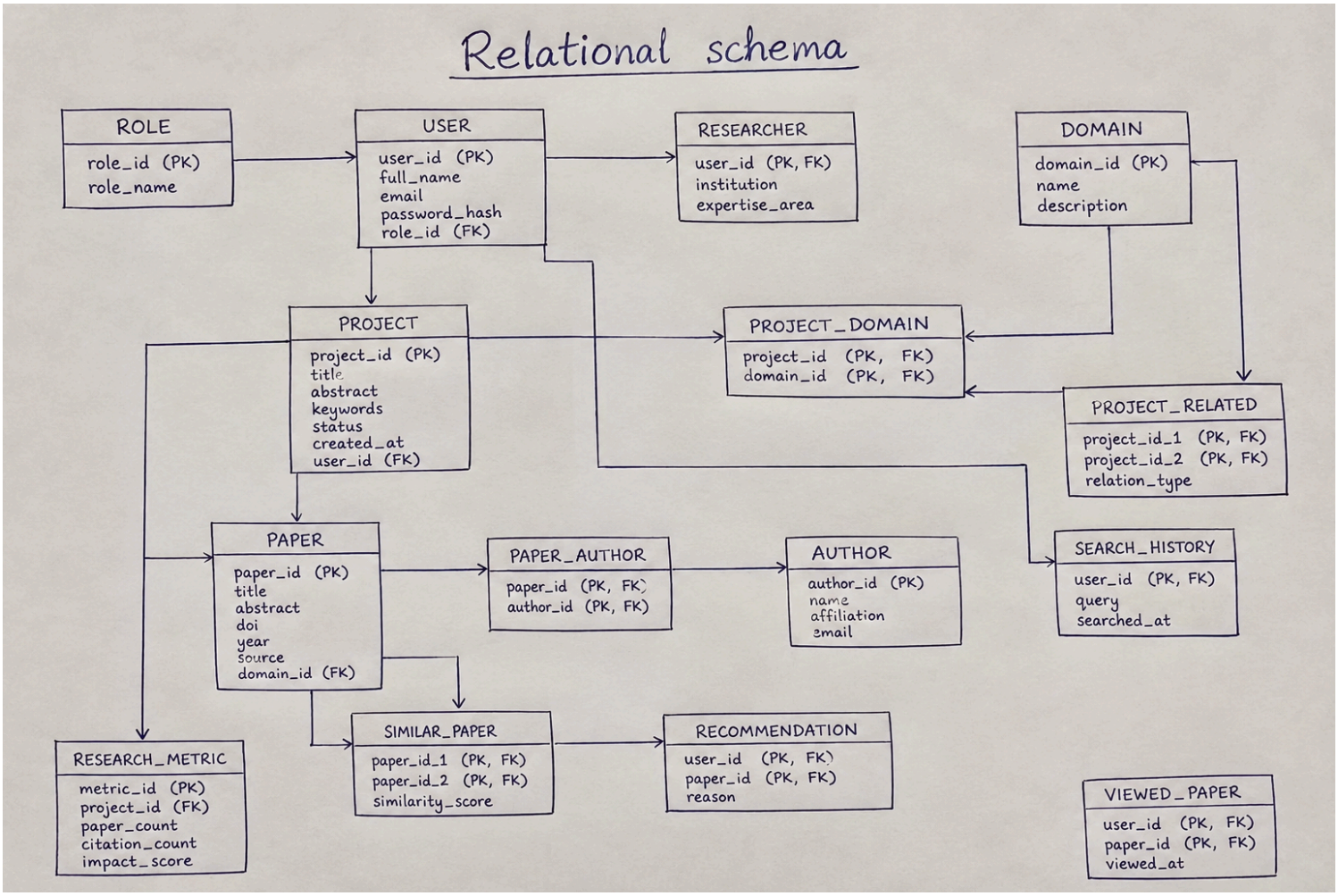
Layer	Technology	Role in ResearchX
Frontend	Next.js	Research dashboard, project analyzer, semantic search and role-based interface.
SQL	MySQL	Users, projects, domains, papers, authors, reviews, audit logs and full-text search.
Vector DB	Qdrant	Project and research-paper embeddings for semantic similarity search.
AI/NLP	Embedding Model	Converts project descriptions, abstracts and research text into vectors.

4. Key DBMS & Intelligent Features

Vector Similarity: semantic project/paper matching using embeddings. **Full-Text Search:** MySQL indexed search over research text. **Triggers:** automatically record project status changes. **Stored Procedures:** controlled project approval operations. **Transactions:** keep approval and review updates atomic. **Indexes:** speed up project and paper lookup.



Relational schema



ResearchX

Intelligent Research Discovery & Project Similarity Platform

DDL commands

ResearchX — PostgreSQL DDL

```sql
-- ============================================================
-- ResearchX - Relational Schema DDL (PostgreSQL)
-- Scope: relational core only (PostgreSQL layer of the architecture)
-- ============================================================

CREATE TABLE role (
    role_id       SMALLINT PRIMARY KEY,
    role_name     VARCHAR(30) NOT NULL UNIQUE,
    CHECK (role_name IN ('Admin','Student','Faculty'))
);

CREATE TABLE app_user (
    user_id       SERIAL PRIMARY KEY,
    username      VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(120) NOT NULL UNIQUE,
    role_id       SMALLINT NOT NULL REFERENCES role(role_id),
    created_at    TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE domain (
    domain_id     SERIAL PRIMARY KEY,
    domain_name   VARCHAR(150) NOT NULL UNIQUE,
    description   TEXT
);

CREATE TABLE project (
    project_id       SERIAL PRIMARY KEY,
    student_id       INTEGER NOT NULL REFERENCES app_user(user_id),
    title            VARCHAR(300) NOT NULL,
    problem_statement TEXT NOT NULL,
    description      TEXT,
    status           VARCHAR(30) NOT NULL DEFAULT 'Pending',
    approved_by      INTEGER REFERENCES app_user(user_id),
    created_at       TIMESTAMP NOT NULL DEFAULT now(),
    CHECK (status IN ('Pending','Approved','Rejected'))
);

CREATE TABLE project_review (
    review_id     SERIAL PRIMARY KEY,
    project_id    INTEGER NOT NULL REFERENCES project(project_id)
                  ON DELETE CASCADE,
    faculty_id    INTEGER NOT NULL REFERENCES app_user(user_id),
    decision      VARCHAR(30) NOT NULL,
    comments      TEXT,
    review_date   TIMESTAMP NOT NULL DEFAULT now(),
    CHECK (decision IN ('Approved','Rejected','Changes Required'))
);

CREATE TABLE author (
    author_id     SERIAL PRIMARY KEY,
    name          VARCHAR(150) NOT NULL,
    affiliation   VARCHAR(200),
    email         VARCHAR(120) UNIQUE
);

CREATE TABLE research_paper (
    paper_id       SERIAL PRIMARY KEY,
    title          VARCHAR(500) NOT NULL,
    abstract       TEXT,
    keywords       TEXT,
    doi            VARCHAR(255) UNIQUE,
    file_url       VARCHAR(500),
    published_in   VARCHAR(255),
    year           SMALLINT,
    cited_by_count INTEGER NOT NULL DEFAULT 0
                   CHECK (cited_by_count >= 0)
);

CREATE TABLE paper_author (
    paper_id      INTEGER NOT NULL REFERENCES research_paper(paper_id)
                  ON DELETE CASCADE,
    author_id     INTEGER NOT NULL REFERENCES author(author_id)
                  ON DELETE CASCADE,
    author_order  SMALLINT,
    PRIMARY KEY (paper_id, author_id)
);
```

ResearchX — DDL commands (continued)

ResearchX — PostgreSQL DDL

```sql
CREATE TABLE citation (
    citation_id      SERIAL PRIMARY KEY,
    citing_paper_id  INTEGER NOT NULL REFERENCES research_paper(paper_id)
                     ON DELETE CASCADE,
    cited_paper_id   INTEGER NOT NULL REFERENCES research_paper(paper_id)
                     ON DELETE CASCADE,
    UNIQUE (citing_paper_id, cited_paper_id),
    CHECK (citing_paper_id <> cited_paper_id)
);

CREATE TABLE saved_item (
    save_id     SERIAL PRIMARY KEY,
    user_id     INTEGER NOT NULL REFERENCES app_user(user_id)
                ON DELETE CASCADE,
    paper_id    INTEGER NOT NULL REFERENCES research_paper(paper_id)
                ON DELETE CASCADE,
    saved_at    TIMESTAMP NOT NULL DEFAULT now(),
    UNIQUE (user_id, paper_id)
);

-- ResearchX semantic-search support
-- Embedding vectors are stored in the vector database;
-- this table keeps the relational reference/metadata.
CREATE TABLE embedding_ref (
    embedding_id SERIAL PRIMARY KEY,
    entity_type  VARCHAR(30) NOT NULL,
    entity_id    INTEGER NOT NULL,
    vector_ref   VARCHAR(150) NOT NULL UNIQUE,
    model        VARCHAR(150) NOT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT now(),
    CHECK (entity_type IN ('PROJECT','RESEARCH_PAPER'))
);

-- Useful relational indexes
CREATE INDEX idx_project_student
    ON project(student_id);

CREATE INDEX idx_project_status
    ON project(status);

CREATE INDEX idx_paper_year
    ON research_paper(year);

CREATE INDEX idx_citation_citing
    ON citation(citing_paper_id);

CREATE INDEX idx_citation_cited
    ON citation(cited_paper_id);

-- Seed the role lookup table
INSERT INTO role (role_id, role_name) VALUES
    (1, 'Admin'),
    (2, 'Student'),
    (3, 'Faculty');
```